# Assignemnt - 2

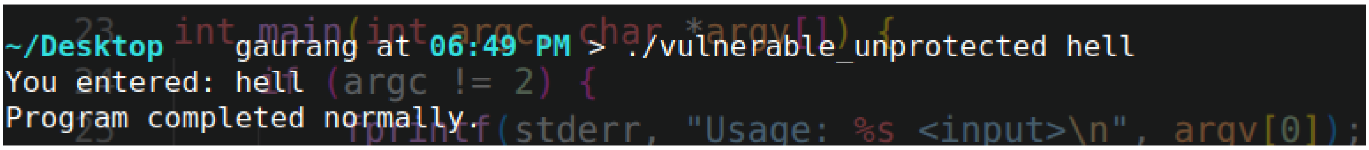
1. Write a small C program with an intentionally unsafe function (e.g., using gets() or strcpy() into a fixed-size stack buffer, with no bounds checking).

> source code

```c
#include <stdio.h>
#include <string.h>

void vulnerable(const char *input) {
    char buffer[5];

    strcpy(buffer, input);

    printf("You entered: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <input>\n", argv[0]);
        return 1;
    }

    vulnerable(argv[1]);
    printf("Program completed normally.\n");

    return 0;
}
```

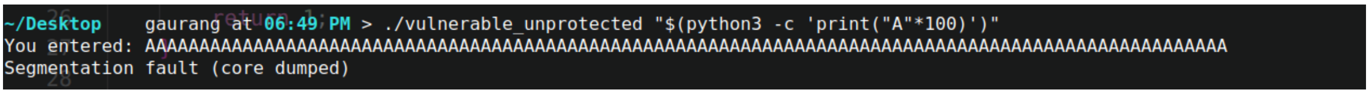
2. Compile it WITHOUT protection and run it with a normal input, then a deliberately oversized input to trigger a crash

> Normal Input



```
~/Desktop    gaurang at 06:49 PM > ./vulnerable_unprotected hell
You entered: hell
Program completed normally.
```

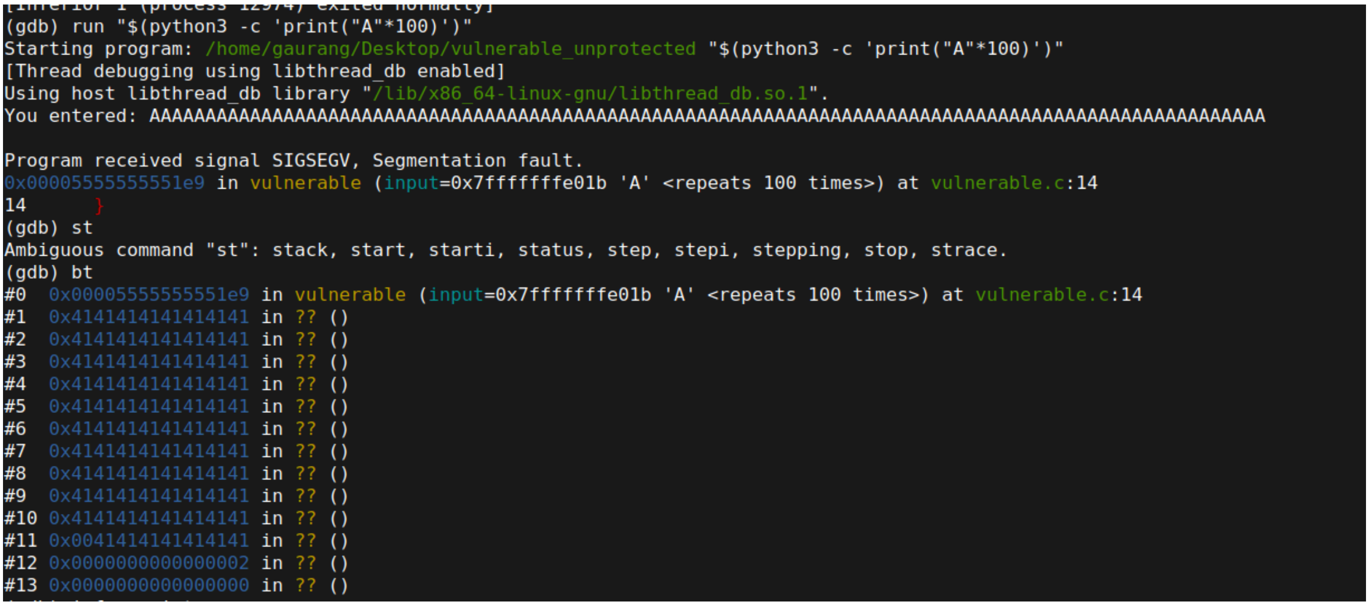
> Oversized Input



```
~/Desktop    gaurang at 06:49 PM > ./vulnerable_unprotected "$(python3 -c 'print("A"*100)')"
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)
```

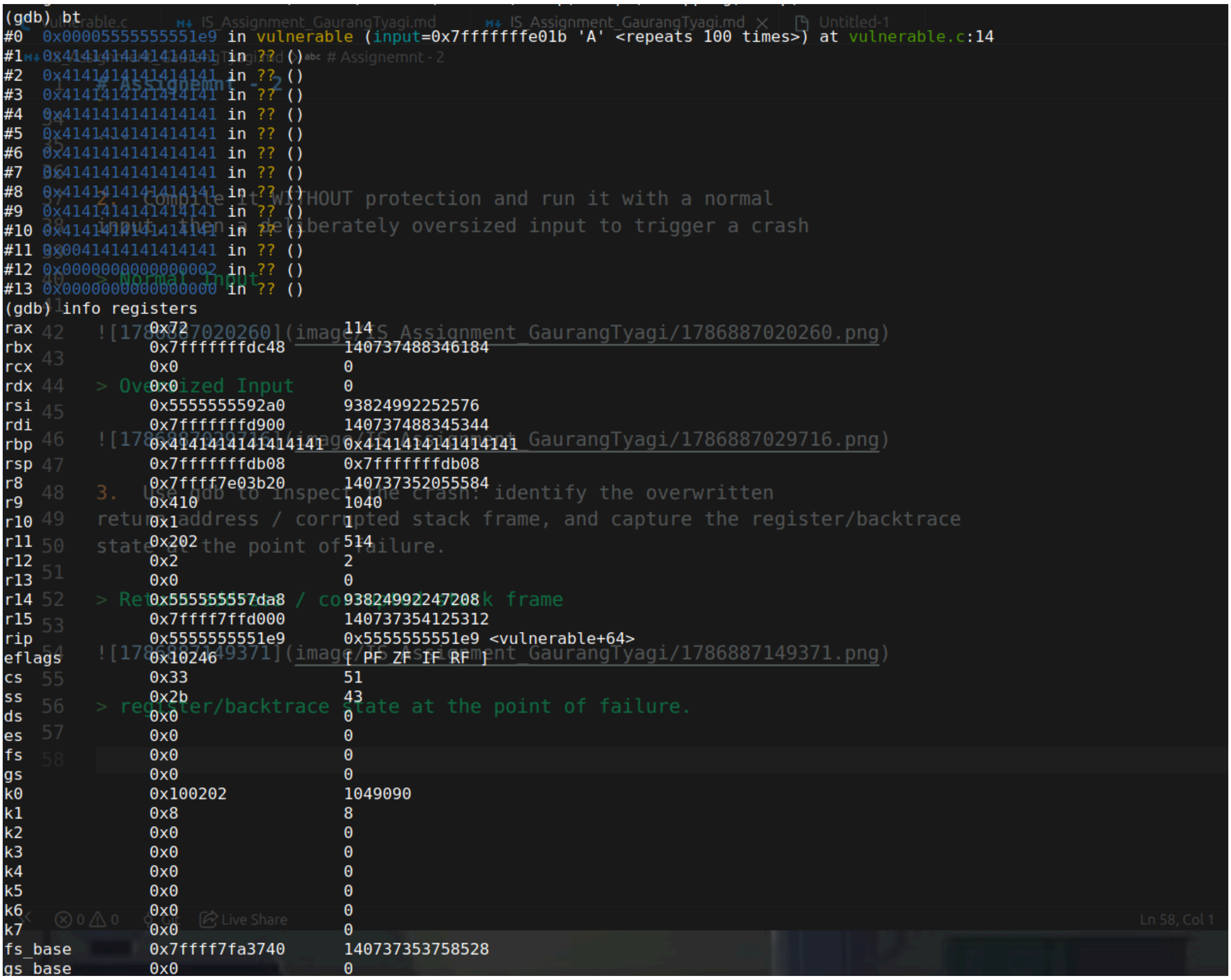
3. Use gdb to inspect the crash: identify the overwritten return address / corrupted stack frame, and capture the register/backtrace state at the point of failure.

> Return address / corrupted stack frame



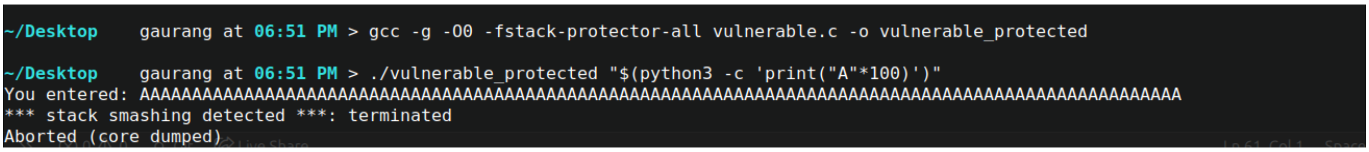
```
(gdb) run "$(python3 -c 'print("A"*100)')"
Starting program: /home/gaurang/Desktop/vulnerable_unprotected "$(python3 -c 'print("A"*100)')"
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0x00005555555551e9 in vulnerable (input=0x7fffffffe01b 'A' <repeats 100 times>) at vulnerable.c:14
14      }
(gdb) st
Ambiguous command "st": stack, start, starti, status, step, stepi, stepping, stop, strace.
(gdb) bt
#0  0x00005555555551e9 in vulnerable (input=0x7fffffffe01b 'A' <repeats 100 times>) at vulnerable.c:14
#1  0x4141414141414141 in ?? ()
#2  0x4141414141414141 in ?? ()
#3  0x4141414141414141 in ?? ()
#4  0x4141414141414141 in ?? ()
#5  0x4141414141414141 in ?? ()
#6  0x4141414141414141 in ?? ()
#7  0x4141414141414141 in ?? ()
#8  0x4141414141414141 in ?? ()
#9  0x4141414141414141 in ?? ()
#10 0x4141414141414141 in ?? ()
#11 0x0041414141414141 in ?? ()
#12 0x0000000000000002 in ?? ()
#13 0x0000000000000000 in ?? ()
```

> register/backtrace state at the point of failure.



```
(gdb) bt
#0  0x00005555555551e9 in vulnerable (input=0x7fffffffe01b 'A' <repeats 100 times>) at vulnerable.c:14
#1  0x4141414141414141 in ?? ()
#2  0x4141414141414141 in ?? ()
#3  0x4141414141414141 in ?? ()
#4  0x4141414141414141 in ?? ()
#5  0x4141414141414141 in ?? ()
#6  0x4141414141414141 in ?? ()
#7  0x4141414141414141 in ?? ()
#8  0x4141414141414141 in ?? ()
#9  0x4141414141414141 in ?? ()
#10 0x4141414141414141 in ?? ()
#11 0x0041414141414141 in ?? ()
#12 0x0000000000000002 in ?? ()
#13 0x0000000000000000 in ?? ()
(gdb) info registers
rax            0x72                114
rbx            0x7fffffffdc48      140737488346184
rcx            0x0                 0
rdx            0x0                 0
rsi            0x5555555592a0      93824992252576
rdi            0x7fffffffd900      140737488345344
rbp            0x4141414141414141  0x4141414141414141
rsp            0x7fffffffdb08      0x7fffffffdb08
r8             0x7ffff7e03b20      140737352055584
r9             0x410               1040
r10            0x1                 1
r11            0x202               514
r12            0x2                 2
r13            0x0                 0
r14            0x555555557da8      93824992247208
r15            0x7ffff7ffd000      140737354125312
rip            0x5555555551e9      0x5555555551e9 <vulnerable+64>
eflags         0x10246             [ PF ZF IF RF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
k0             0x100202            1049090
k1             0x8                 8
k2             0x0                 0
k3             0x0                 0
k4             0x0                 0
k5             0x0                 0
k6             0x0                 0
k7             0x0                 0
fs_base        0x7ffff7fa3740      140737353758528
gs_base        0x0                 0
```

4. Recompile the SAME program WITH protections enabled. Repeat the oversized input and show that the program now terminates safely



```
~/Desktop    gaurang at 06:51 PM > gcc -g -O0 -fstack-protector-all vulnerable.c -o vulnerable_protected

~/Desktop    gaurang at 06:51 PM > ./vulnerable_protected "$(python3 -c 'print("A"*100)')"
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
*** stack smashing detected ***: terminated
Aborted (core dumped)
```